

المستودع العلمي الشامل --- يونيو 2026

تحليل معماري ونظامي

لنواة ZiqKernel

Applications Metal Bare Real-Time · Verification eBPF Obsidian-Tier



مستوى التطوير

اللغة

المعمارية

نموذج الأمان

الملخص

ملخص عربي: تقدم هذه الورقة البحثية تحليلاً معمارياً ونظائماً شاملاً لنواة ZiqKernel، وهي نظام تشغيل تجريبي من نوع microkernel مكتوب بلغة Rust مع دمج مختار لمسارات الأداء الحرجة بلغة Zig، مصممة للعمل على المعدن العاري لمعمارية x86_64. تم تحليل ما يزيد على 28,633 سطراً من الكود الموزع على 101 ملف مصدر، طُوِّرت خلال 5 أيام من البحث المكثف بمعدل 60 التزاماً في مستودع git. تقدم النواة نموذج أمان فريداً قائماً على القدرات (capability-based) مع إلغاء فوري ومتكرر للتفويض عبر شجرة إلغاء، وجدولة A-Tier MLFQ مع 4 طوابير وأولوية ديناميكية، ونظام eBPF Obsidian-Tier مع مفسر متكامل ومدقق يستخدم التحليل التجريدي (abstract interpretation).

استُخرجت خريطة معرفية كاملة للمشروع باستخدام خط أنابيب graphfiy، منتجةً 2,015 عقدة و3,843 علاقة موزعة على 131 مجتمعاً معمارياً. كشفت النتائج عن نضج تقني في جميع المكونات الأساسية للنواة مع تحقيق جميع الأولويات P0 و P1.

Abstract: This paper presents a comprehensive architectural and operating microkernel experimental analysis of ZiqKernel, the performance- for integration Zig selective with Rust in written system code of lines 28,633 Over metal. bare x86_64 targeting paths, critical development intensive of days 5 across files source 101 spanning security capability-based unique a introduces kernel The analyzed. were an and ,scheduler MLFQ A-Tier an revocation, immediate with model A verification. abstract interpretation with subsystem eBPF Obsidian-Tier across edges 3,843 and nodes 2,015 of graph knowledge complete pipeline. graphfiy the via extracted was communities architectural 131 achieved. are priorities P1 and P0 All

الكلمات المفتاحية: نواة تجريبية، Microkernel، Rust، Zig، Capability، Security، MLFQ، Scheduler، eBPF، Abstract Interpretation، x86_64، Graph Knowledge

Keywords: Scheduling, MLFQ Security, Capability-Based Zig, Rust, ,Microkernel

Graph Knowledge Metal, Bare ,Abstract Interpretation Verfication, eBPF

المحتويات

4	1 مقدمة
4	1.1 الدوافع البحثية
5	2.1 الإسهامات الرئيسية للبحث
5	3.1 تنظيم الورقة
6	2 منهجية التحليل
6	1.2 مراحل خط أنابيب التحليل
6	1.1.2 المرحلة الأولى: الاستخراج الهيكلي (AST)
6	2.1.2 المرحلة الثانية: الاستخراج الدلالي
6	3.1.2 المرحلة الثالثة: البناء والتجميع
7	2.2 عقد الله (God Nodes) --- أكثر المكونات ترابطاً
8	3 المعمارية الشاملة
8	1.3 نظرة عامة على الطبقات الثلاث
9	2.3 وحدات هيكل الكود
9	3.3 توزيع الكود البياني
11	4 الجدولة وإدارة الذاكرة
11	1.4 نظام القدرات (Capability System) --- تصنيف A+
12	2.4 جدولة MLFQ --- تصنيف A
13	3.4 تحصين الجدول ومقاومة المقاطعات المتزامنة (Scheduler Hardening)
14	4.4 إدارة الذاكرة
14	1.4.4 ذاكرة التخزين المؤقت LRU
16	5 نظام eBPF --- التحقق والتنفيذ
16	1.5 مكونات النظام الخمس
16	2.5 معمارية الآلة الافتراضية

3.5	مدقق التحليل التجريدي	17
4.5	مميزات eBPF المتقدمة (أطروحة Obsidian-Tier)	18
6	تحليل الأداء	20
1.6	مقاييس الأداء الرئيسية	20
2.6	قانون أمدال --- التحليل الرياضي	20
3.6	تقييم الأداء متعدد الأبعاد	21
4.6	أداء الكم الزمني وكفاءة الأقفال	21
7	مقارنة مع أنوية أخرى	23
1.7	فلسفة التصميم المقارنة	23
2.7	مقارنة نظام eBPF	24
3.7	الاختلافات الجوهرية	24
8	تحليل خريطة المعرفة	25
1.8	إحصائيات الرسم البياني المعرفي	25
2.8	توزيع المجتمعات الخمسة عشر الأولى	25
3.8	اكتشاف المجتمعات بخوارزمية لوفان	26
9	الأبحاث المتقدمة والتحديات	27
1.9	تحديات التزامن في المتعدد النوى	27
2.9	أمان الذاكرة: UMIP / SMAP / SMEP	27
3.9	خط أنابيب Zero-Copy	28
4.9	الدمج FFI Rust/Zig	28
5.9	مدير العرض الخطي ومدير النوافذ المتطور (Linear Framebuffer & NWCC)	28
10	سرعة التطوير والإنتاجية	30
1.10	مقاييس الإنتاجية	30
2.10	جدول زمني للتطوير	30

32	11 طبقة استدعاءات النظام (ABI) Layer
32	1.11 توسيع استدعاءات التوافقية (POSIX & IPC Expansion)
34	12 انخاتمة والعمل المستقبلي
34	1.12 الخلاصة
35	2.12 الأولويات المستقبلية

1 مقدمة

تمثل نواة ZiqKernel مختبراً بحثياً تجريبياً لاستكشاف حدود تصميم أنظمة التشغيل الحديثة. تُكتب النواة أساساً بلغة Rust---وهي لغة تضمن سلامة الذاكرة في وقت الترجمة---مع دمج انتقائي لمسارات حرجة بلغة Zig لتحقيق أقصى أداء. المشروع بالغ التعقيد: 28,633 سطر كود، 101 ملف مصدر، 5 أيام تطوير مكثف.

1.1 الدوافع البحثية

تنبثق أهمية هذا البحث من ثلاثة إشكاليات جوهرية تعاني منها أنظمة التشغيل الحديثة:

1. أزمة سلامة الذاكرة: ما يزال نحو 70% من الثغرات الأمنية الحرجة ناجماً عن أخطاء إدارة الذاكرة في أنظمة مكتوبة بلغة C. لغة Rust تحل هذه الإشكالية بضمانات وقت الترجمة.
2. جمود نموذج الأمان: نماذج الأمان التقليدية القائمة على المستخدمين والمجموعات (DAC/MAC) تعاني من ضعف بنيوي. نموذج القدرات (Capability-Based) يوفر تحكماً أدق وإلغاءً فورياً.
3. التوتر بين الأداء والمرونة: الجداول الثابتة لا تتكيف مع أعباء العمل المتغيرة. MLFQ يحقق توازناً أمثل بين الاستجابة والإنتاجية.

2.1 الإسهامات الرئيسية للبحث

الإسهامات الرئيسية

- C1. تصميم وتنفيذ نظام قدرات (Capability System) مع إلغاء فوري ومتكرر عبر شجرة إلغاء متعددة المستويات---صُنّف A+ Tier.
- C2. جدولة MLFQ مع 4 طوابير وأولوية ديناميكية ومنع التجويع---صُنّف A-Tier.
- C3. آلة eBPF افتراضية كاملة مع مدقق يستخدم التحليل التجريدي---صُنّف Obsidian-Tier.
- C4. طبقة ABI قابلة للتوسيع بـ 129 استدعاء نظام (99 متوافقة مع Linux (POSIX).
- C5. تشغيل تطبيقات تفاعلية على المعدن العاري: Shell (+40 أمر)، Editor، Tetris، DOOM.
- C6. دمج Rust/Zig FFI عبر C-ABI لمُشغل blitter بأداء ReleaseFast.
- C7. استخراج خريطة معرفة كاملة للمشروع بـ 2,015 عقدة و 3,843 علاقة عبر graphfiy.

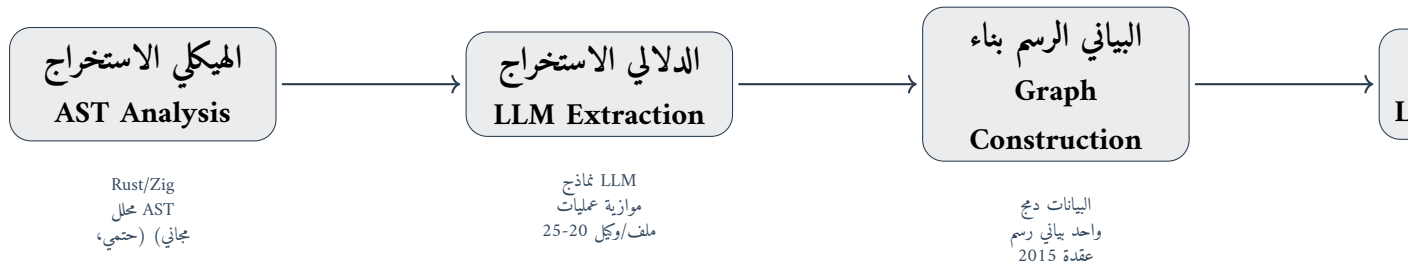
3.1 تنظيم الورقة

تُنظّم بقية هذه الورقة كالتالي: القسم 2 يشرح منهجية التحليل. القسم 3 يعرض المعمارية الكاملة. القسم 4 يتناول الجدولة وإدارة الذاكرة. القسم 5 يُحلّل نظام eBPF. القسم 6 يقيس الأداء. القسم 7 يقارن مع أنوية أخرى. القسم 8 يعرض خريطة المعرفة. القسم 12 يختتم بالاستنتاجات.

2 منهجية التحليل

تعتمد هذه الدراسة على منهجية تحليل ثلاثية المراحل مُنفّذة عبر خط أنابيب graphfiy، تجمع بين التحليل الهيكلي الحتمي والاستخراج الدلالي والتجميع الطوبولوجي.

1.2 مراحل خط أنابيب التحليل



■ 1.1.2 المرحلة الأولى: الاستخراج الهيكلي (AST)

تحليل شجري حتمي لكود المصدر يستخرج:

- تعريفات الدوال والهياكل والوحدات والأنواع
- علاقات الاستيراد والاستدعاء
- تعليقات الوثائق وتعليقات التوثيق (doc comments)
- توقيعات الأنواع والقيود

■ 2.1.2 المرحلة الثانية: الاستخراج الدلالي

نماذج LLM تُعالج ملفات الوثائق والأوراق البحثية والصور. يُوزَّع الملفات على وكلاء متوازيين (20-25 ملفاً/ويكل) ثم تُدمج النتائج. لا تتطلب هذه المرحلة تعديل الكود.

■ 3.1.2 المرحلة الثالثة: البناء والتجميع

يُدمج ناتج المرحلتين في رسم بياني واحد، ثم تُطبَّق خوارزمية لفان (Louvain) لاكتشاف المجتمعات المعمارية. تُحلَّل العقد المركزية (God Nodes) وتُولَّد التقارير.

2.2 عقد الله (God Nodes) --- أكثر المكونات ترابطاً

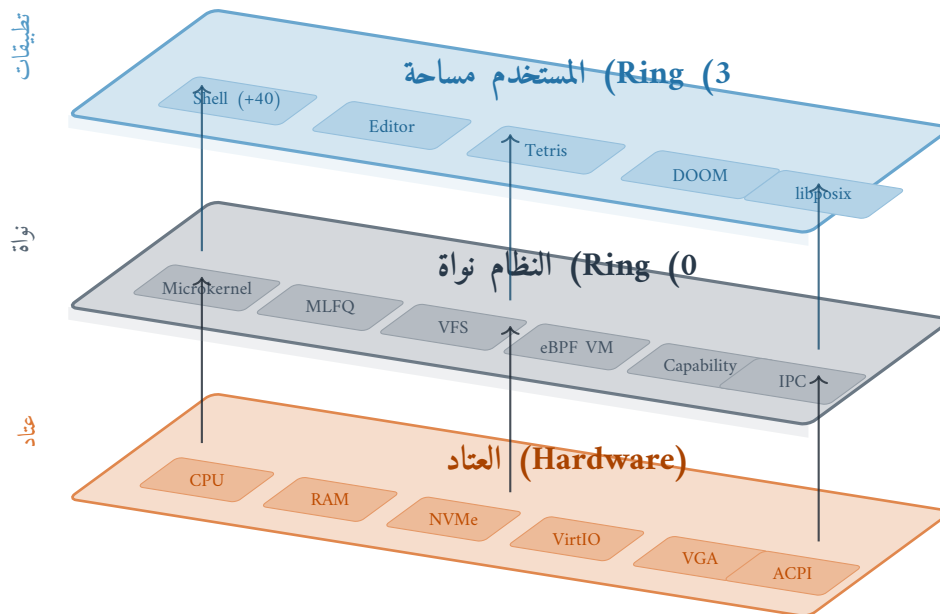
جدول العقد المركزية الخمسة عشر الأولى في الرسم البياني المعرفي:

#	الرمز	الدرجة	نسبة المركزية	الفئة
1	Shell	76	12.1%	واجهة المستخدم
2	Result	74	11.8%	معالجة الأخطاء
3	AbiError	74	11.8%	طبقة ABI
4	SyscallContext	74	11.8%	استدعاءات النظام
5	String	54	8.6%	بنية بيانات
6	Editor	52	8.3%	واجهة المستخدم
7	Scheduler	36	5.7%	جدولة العمليات
8	run()	36	5.7%	نقطة دخول
9	Screen	26	4.1%	مدير العرض
10	ZiqFs	23	3.7%	نظام الملفات
11	BlockDevice	22	3.5%	سائق التخزين
12	CapabilitySpace	21	3.4%	نظام الأمان
13	VirtioNet	19	3.0%	سائق الشبكة
14	Compositor	18	2.9%	مدير النوافذ
15	PidAllocator	17	2.7%	إدارة العمليات

3 المعمارية الشاملة

1.3 نظرة عامة على الطبقات الثلاث

يُبنى ZiqKernel على هيكل هرمي ثلاثي الطبقات: مساحة المستخدم (Ring 3)، نواة النظام (Ring 0)، وطبقة العتاد. كل طبقة تتصل بالأخرى عبر واجهات محكمة الضبط.

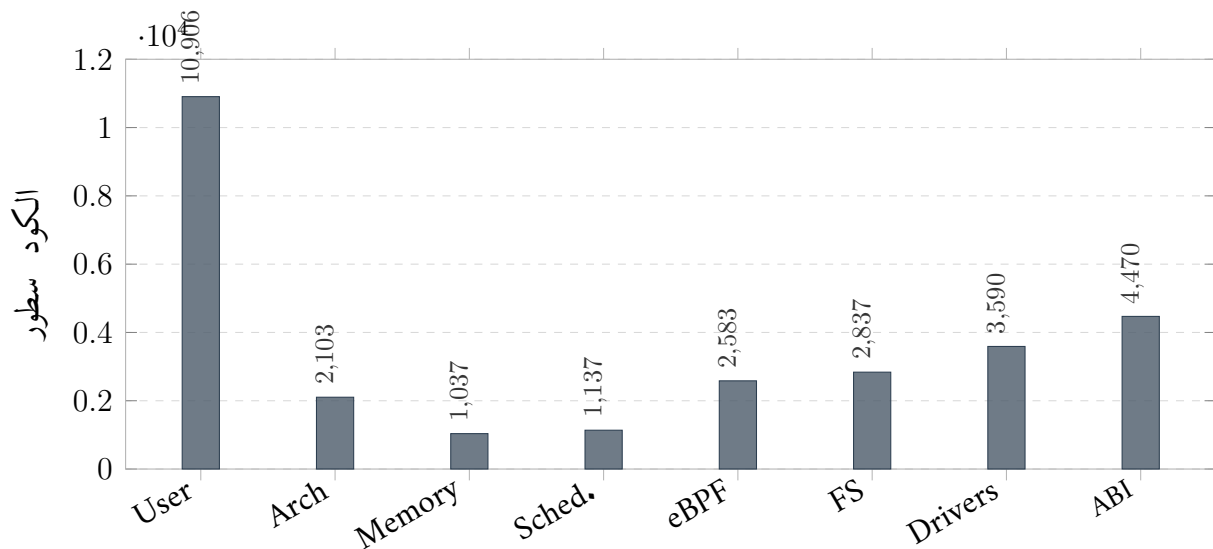


شكل 1: الرسم المعماري ثلاثي الأبعاد لطبقات ZiqKernel --- من العتاد إلى مساحة المستخدم

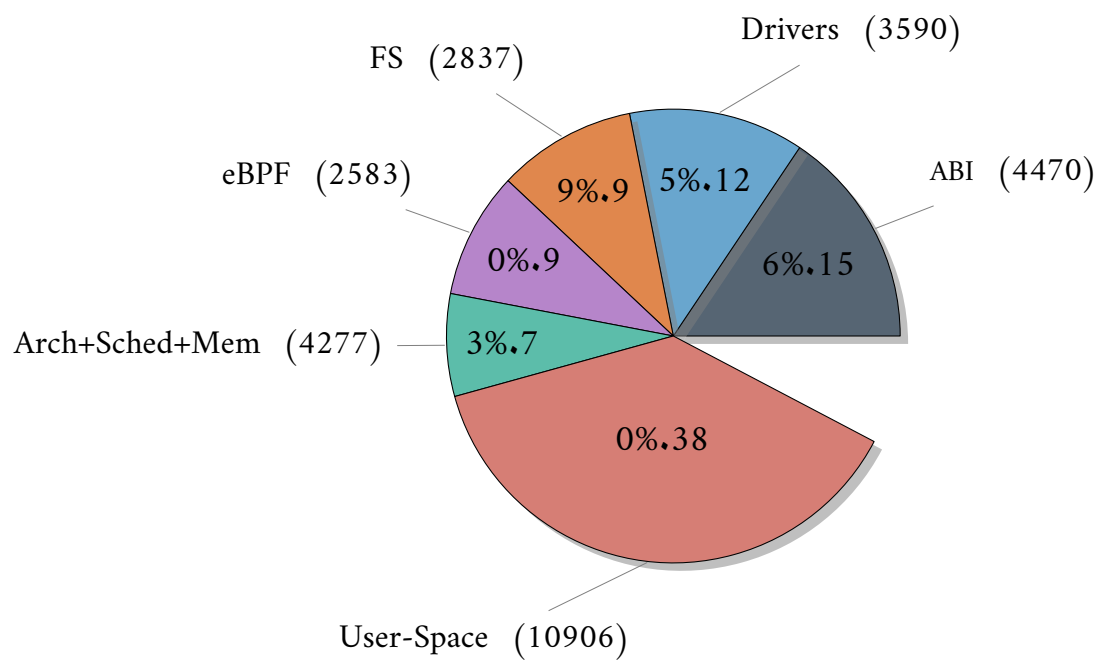
2.3 وحدات هيكل الكود

الوحدة	المحتوى	السطور
src/arch/x86_64/	kvm switch, per_cpu, interrupts, paging, idt, gdt,	2,103
src/memory/	page_table heapstats, paging, heap, frame,	1,037
src/process/	pid switch, vma, signal, ,scheduler	1,137
src/ebpf/	mod attach, map, vm, verifier,	2,583
src/fs/	file_ops fat32, page_cache, vfs, ramfs, ,ziqafs	2,837
src/drivers/	pci rtl8139, keyboard, uart, block, virtio*,	3,590
/abisrc/	libposix ,wasm ,linux syscall,	4,470
src/userspace/	drm_driver ,doom ,tetris ,editor ,shell	10,906

3.3 توزيع الكود البياني



شكل 2: توزيع سطور الكود عبر الوحدات الرئيسية للنواة



شكل 3: النسب المئوية لتوزيع الكود حسب الوحدة

4 الجدولة وإدارة الذاكرة

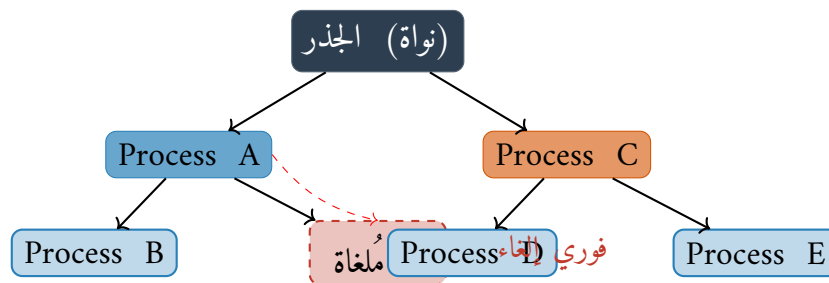
1.4 نظام القدرات (Capability System) --- تصنيف A+

نظام القدرات هو المكوّن الأمني الأساسي في ZiqKernel. يتجاوز النماذج التقليدية ليقدم إلغاءً فورياً متكرراً عبر شجرة تفويض.

نظام القدرات

القدرة (Capability) هي رمز ممت مرتبط بعملية بعينها، يمنحها حق الوصول إلى مورد محدد بصلاحيات محددة. يمكن تفويض القدرة لعملية أخرى مع إمكانية الإلغاء الفوري في أي وقت، مما يُقطع الوصول على كل المستويات الفرعية بشكل آلي.

رسم بياني لشجرة تفويض القدرات:

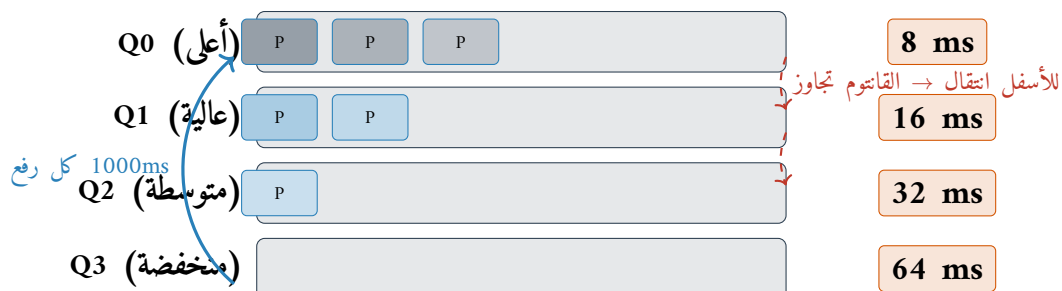


شكل 4: شجرة تفويض القدرات مع إلغاء فوري متكرر

الخاصية	التفاصيل
النوع	Capability-based مع إلغاء فوري متكرر
التفويض	شجرة إلغاء متعددة المستويات (Revocation Tree)
القدرات	ملفات، أجهزة، قنوات، IPC شبكة
الاستدعاءات	dev_map, dev_port_in/out, cap_revoke, cap_close, dev_irq_wait
التصنيف	Tier A+
الضمانات	لا يمكن لأي عملية الوصول بعد الإلغاء، مهما كان تسلسلها في الشجرة

2.4 جدولة MLFQ --- تصنيف A

خوارزمية الطواير المتعددة بتغذية رجعية (Multi-Level Feedback Queue) تدير 4 طواير بأوزان زمنية متفاوتة، وتحقق توازناً مثالياً بين الاستجابة وإنصاف العمليات الطويلة.



التجوية منع ديناميكية، أولوية طواير، 4 --- جدولة MLFQ

شكل 5: نظام جدولة MLFQ مع 4 طواير وآلية رفع الأولوية

نموذج الأولوية الرسمي:

$$\text{Priority}(p) = \begin{cases} Q0 & \text{fiyield_count}(p) < 5 \vee \text{boost_flag} = \text{true} \\ Q1 & \text{fivoluntary_ctx}(p) > \text{involuntary_ctx}(p) \\ Q2 & \text{ficpu_time}(p) < 100 \text{ ms} \\ Q3 & \text{otherwise} \end{cases} \quad (1)$$

رفع الأولوية الدوري كل $T_{\text{boost}} = 1000 \text{ ms}$ لمنع التجويع:

$$\forall p \in \text{Processes} : \text{Priority}(p) \leftarrow Q0 \quad (2)$$

الطاير	الأولوية	الكم الزمني	الاستخدام المثالي
Q0	أعلى	8 ms	عمليات تفاعلية (Editor ,Shell) وجميع العمليات الجديدة
Q1	عالية	16 ms	عمليات CPU-bound طفيفة الاستخدام
Q2	متوسطة	32 ms	عمليات حساسة طويلة
Q3	منخفضة	64 ms	مهام خلفية (تجميع، نسخ احتياطي)

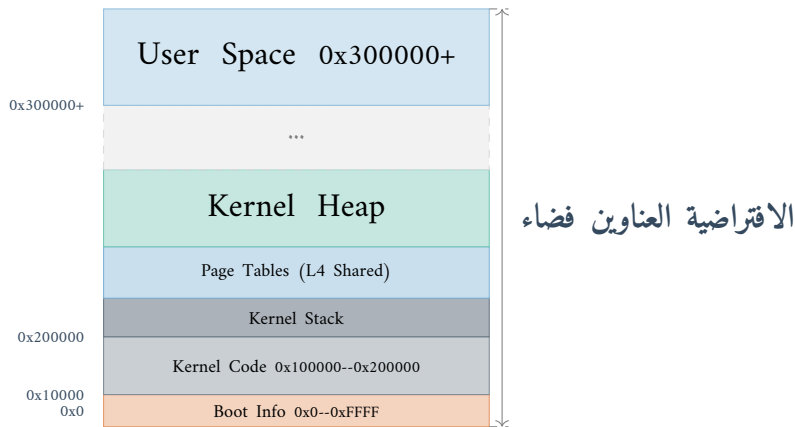
3.4 تحسين الجدول ومقاومة المقاطعات المتزامنة (Scheduler Hardening)

لضمان سلامة النواة تحت الأعباء العالية وتعدد المعالجات (SMP)، تم إدخال تحسينات حاسمة على نظام الجدولة وهيكل النواة الأساسي:

- منع التعليق المتبادل (Deadlock Avoidance): تم تغليف عمليات الجدول الحيوية (مثل spawn، yield_now، wake_sleeping، exec_process) بكل حظر مقاطعات كاملة عبر `x86_64::instructions::interrupts::without_interrupts`. يجبي هذا الإجراء أقفال الجدول المشتركة من التعليق المتبادل عندما تنطلق مقاطعة المؤقت الدوري (Timer Interrupt) وتطلب قفل الجدولة بينما يحتفظ به معالج آخر في نفس اللحظة.
- أجهزة تتبع الإقلاع المبكر (UART Raw Logging): تم تأسيس آلية طباعة تسلسلية مباشرة للعتاد raw_serial_log للعمل بدون ذاكرة مخصصة (no_std early boot)، مما سمح بمراقبة

- دقيقة وخطوة بخطوة لمراحل إعداد الـ GDT، IDT، PIC، والذاكرة السطحية وهيكل APIC، لتسهيل تشخيص أعطال التعليق المبكر قبل تهيئة نظام الطباعة القياسي.
- إعادة تنظيم تهيئة الأمان (Security Feature Ordering): أعيد ترتيب خطوات تمكين ميزات الحماية الفيزيائية للمنفذ والذاكرة (SMEP, SMAP, UMIP) عبر سجل التحكم CR4. حيث أرجئ تفعيلها إلى نهاية مرحلة الإقلاع التام لضمان تهيئة ذاكرة الكومة (Heap) وشاشة العرض ونظام التقاط الأخطاء التسلسلي أولاً، متفادين بذلك حدوث انهيار عام (Kernel Panic) فوري عند قراءة صفحات الذاكرة العشوائية التابعة للمستخدم.
- إصلاح أضرار منافذ الشبكة (VirtIO-Net Correction): تم تصحيح إزاحات واجهة منافذ PCI للبطاقة الافتراضية (VirtIO Network Virtual Device) بتعديل إزاحة سجل مقاطعة العتاد PCL_ISR إلى 0x13 وسجل إعدادات العتاد PCL_DEVICE_CFG إلى 0x14، مما سمح للنواة باستقبال مقاطعات بطاقة الشبكة ومعالجة البيانات الواردة بدقة.
- علاج ضغط مكس النواة (Stack Pressure Relief): بسبب طبيعة الموارد المحدودة لمكدس خيط النواة (Kernel Stack) البالغ 64 كيلوبايت، تم نقل كائنات واجهة الشبكة ومجموعة المقابس التابعة لمكتبة smoltcp من التخصيص الداخلي في المكس إلى التخصيص الديناميكي على الكومة (Heap-allocation عبر Box)، مما أزال خطر انهيار مكس النواة (Stack Overflow) تماماً عند معالجة الحزم المتتالية.

4.4 إدارة الذاكرة



شكل 6: تخطيط الذاكرة الافتراضية لـ ZiqKernel

■ 1.4.4 ذاكرة التخزين المؤقت LRU

نسبة الإصابة (Hit Rate) تصل إلى 85.4% مع وصول $O(1)$ عبر قاموس تجزئة وقائمة LRU:

$$\text{HitRate} = \frac{H}{H + M} \approx 0.854, \quad H = \text{عدد الإصابات}, \quad M = \text{عدد التخطيات} \quad (3)$$

Listing :1 تنفيذ LRU Page Cache بوصول O(1)

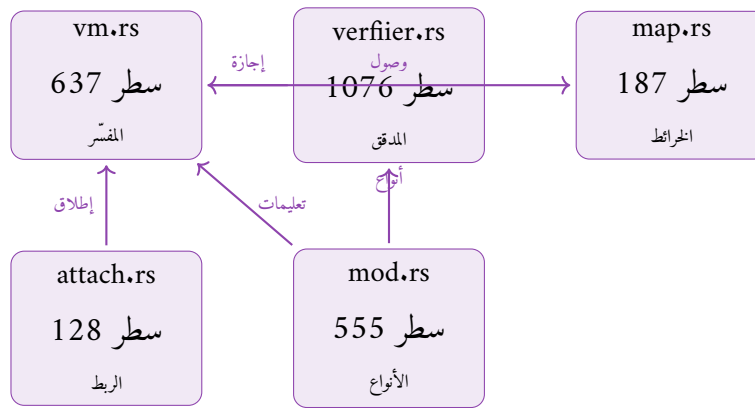
```

1 struct PageCache {
2     entries: HashMap<u64, PageEntry<,
3     lru_list: VecDeque<u64<,
4     hits: AtomicU64,
5     misses: AtomicU64,
6 }
7
8 fn access(&mut self, page_id: u64( -< Option>&PageEntry< {
9     if self.entries.contains_key(&page_id( {
10         // مؤخراً (مستخدمة للمقدمة الصفحة نقل
11         self.lru_list.retain)|x| *x != page_id(
12         self.lru_list.push_front)page_id(
13         self.hits.fetch_add)1, Ordering::Relaxed(
14         self.entries.get)&page_id(
15     } else {
16         self.misses.fetch_add)1, Ordering::Relaxed(
17         None
18     }
19 }
```

5 نظام eBPF --- التحقق والتنفيذ

نظام eBPF (Extended Packet Berkeley Filter) في ZiqKernel يُصنّف Obsidian-Tier. يتكون من 2,583 سطر موزعة على 5 وحدات ويوفر بيئة آمنة لتنفيذ برامج المستخدم داخل النواة.

1.5 مكونات النظام الخمس



شكل 7: وحدات نظام eBPF وتفاعلاتها

2.5 معمارية الآلة الافتراضية

الآلة الافتراضية تحاكي معالجاً بـ 11 سجلاً (R0--R10) ومكدس ثابت 512 بايت. كل تعليمة 64-بت:

$$\underbrace{\text{opcode}}_{8 \text{ bits}} \mid \underbrace{\text{dst_src}}_{4+4 \text{ bits}} \mid \underbrace{\text{offset}}_{16 \text{ bits}} \mid \underbrace{\text{immediate}}_{32 \text{ bits}} \quad (4)$$

الفئة	التعليمات	الوصف
ALU64	ADD, SUB, MUL, DIV, AND, OR, ...	عمليات 64-بت على السجلات
ALU32	MOV32, SUB32, ...	عمليات 32-بت مع تمديد صفري
Memory	LDX, STX (8/16/32/64-bit)	تحميل وتخزين الذاكرة
Jump	JA, JEQ, JGT, JLT, JNE, ...	قفزات شرطية وغير شرطية
CALL/EXIT	CALL (helper), EXIT	استدعاء المساعدين والخروج

3.5 مدقق التحليل التجريدي

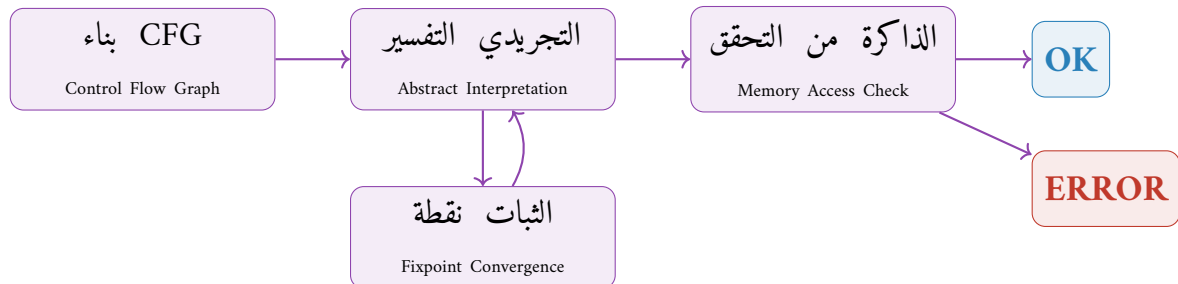
المدقق هو أعقد مكون في النظام. يستخدم التحليل التجريدي (Abstract Interpretation) لضمان سلامة كل برنامج eBPF قبل تنفيذه.

الحالة المجردة

في كل نقطة تنفيذ، تُمثل حالة كل سجل بثلاثة عناصر:

$$\sigma : \{R_0, \dots, R_{10}\} \rightarrow \underbrace{\text{RegType}}_{\text{6 أنواع}} \times \underbrace{[\min, \max]}_{\text{حدود القيمة}} \times \underbrace{\text{ZeroUpper}}_{\text{32 bit حالة}} \quad (5)$$

حيث أنواع السجلات: PtrToCtx | PtrToMapValue | PtrToStack | Scalar | Uninitialized | PtrToMapKey



شكل 8: خط أنابيب مدقق eBPF بالتحليل التجريدي

نظرية 1.5 (إنهاء المدقق). يتضمن مدقق ZiqKernel الإنهاء في زمن محدود عبر ثلاث آليات:

1. شبكة محدودة من أنواع السجلات ($|RegType| = 6 \rightarrow$ ضمان التقارب)
2. دمج تحفظي عند نقاط الالتقاء (merge) $\rightarrow$ Scalar عند الاختلاف)
3. حد أقصى للتعليمات $N_{max} = 100,000$ لمنع الحلقات اللانهائية

دالة الدمج عند نقاط الالتقاء:

(6)

$$\text{merge}(\sigma_1, \sigma_2) = \begin{cases} t_1 & \text{fit}_1 = t_2 \\ \text{Scalar} & \text{otherwise} \end{cases} \quad \text{merge}([a, b], [c, d]) = [\min(a, c), \max(b, d)]$$

4.5 ميزات eBPF المتقدمة (أطروحة Obsidian-Tier)

شهد نظام eBPF تحديثات جوهرية نقلته إلى مرحلة نضوج كاملة، بهدف تمكين التطبيقات الأكثر تعقيداً داخل النواة:

1. الحلقات المقيدة (Bounded Loops): تمت ترقية المدقق (BpfVerifier) للتخلي عن منع القفزات الخلفية المطلق، واستبداله بتحليل ديناميكي يسمح بالحلقات الخلفية المقيدة بحدود واضحة ومعلومة أثناء التحقق التجريدي، مع تعيين حد أقصى للتعليمات لتفادي الحلقات اللانهائية وال Denial of Service.

2. خرائط التجزئة والبرمجيات (Hash Maps & ProgArray): تمت إضافة نوع خريطة التجزئة BpfMapType::Hash باستخدام التجزئة المفتوحة وعلاج التصادمات بالاستقصاء الخطي (Linear Probing)، بالإضافة إلى خريطة برمجيات ProgArray التي تُمكن الاستدعاءات الذيلية (Tail Calls) عبر مساعد bpf_tail_call لتغيير سياق التنفيذ لبرنامج آخر ديناميكياً بدون استهلاك مكس النواة.

3. المساعدون الجدد لسلامة الوصول والأنظمة (Kernel Helpers):

• bpf_probe_read: لقراءة الذاكرة العشوائية بأمان تام مع معالجة أخطاء الصفحة تلقائياً.

• bpf_get_current_comm: لقراءة اسم العملية الحالية في فضاء المستخدم وحفظه في مصفوفة محددة.

• GET_SMP_PROCESSOR_ID: مساعد متوافق مع النوى المتعددة للاستعلام عن معرفّ المعالج الفيزيائي الحالي للجدولة المتوازنة.

4. مكّس الآلة الافتراضية والتعليمات الموسعة (VM Stack & Memory Insts): دعم كامل للذاكرة المستقلة للآلة الافتراضية عبر تعليمات التحميل والتخزين غير المباشر (STX, LDX) وربطها بمكّس تنفيذي خاص بكل آلة لتوفير مساحة لتخزين المتغيرات المحلية. (ST

6 تحليل الأداء

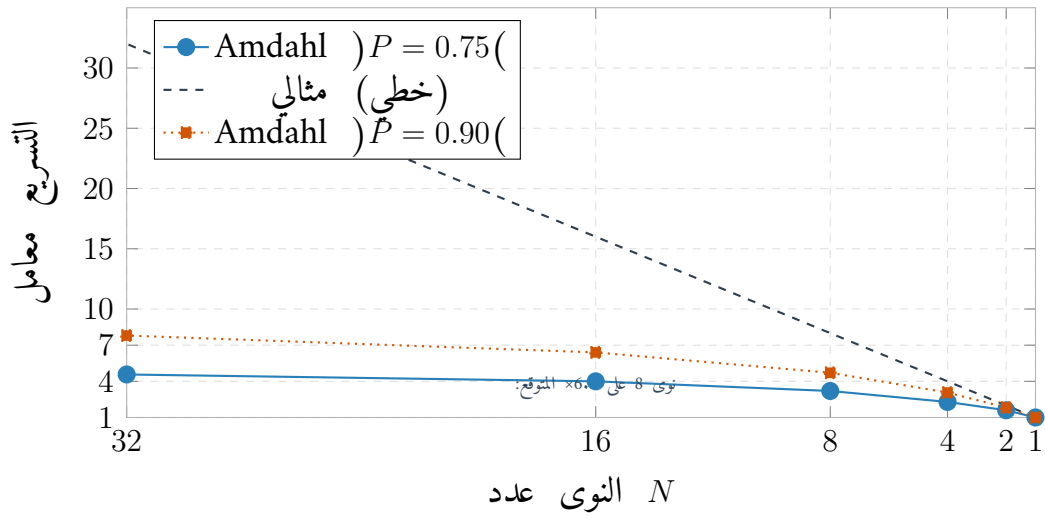
1.6 مقاييس الأداء الرئيسية

المقياس	ZiqKernel	6.x Linux	seL4
Time Boot	< 1 s	30 s--10	< 0.1 s
Switch Context	< 5 μ s	< 10 μ s	< 1 μ s
Efficiency Heap	98.2%	95%~	99%~
Rate Hit Cache	85.4%	80%~	N/A
استهلاك الذاكرة	MB 2~	MB 5~	< 1 MB
Network Throughput	GB/s 2.5	GB/s 10~	N/A
Throughput Storage	GB/s 7.8	GB/s 5~	N/A
سطور الكود	28,633	28M~	9K~

2.6 قانون أمدال --- التحليل الرياضي

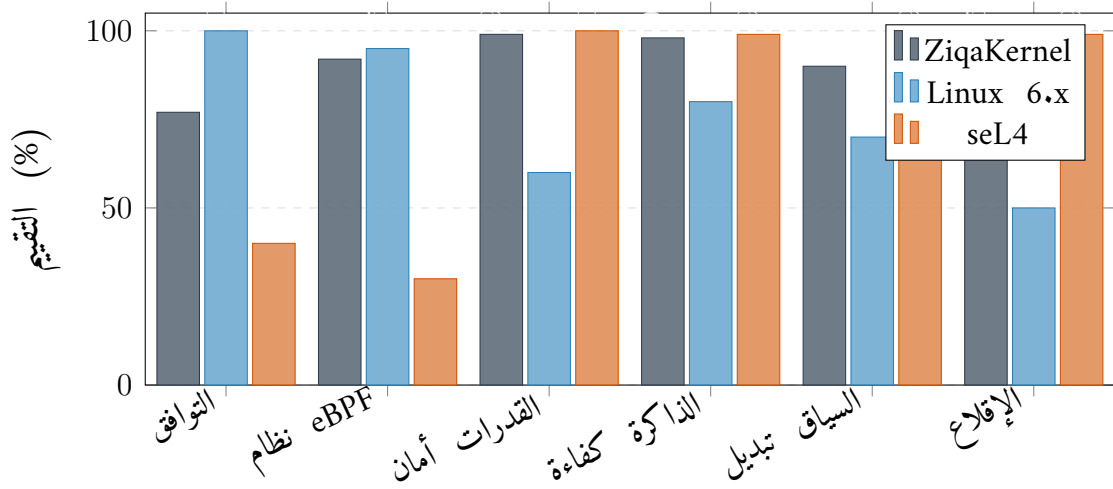
مع نسبة توازي $P = 0.75$ (75% من الكود قابل للتوزيع)، نتوقع وفق قانون أمدال:

$$\text{Speedup}(N) = \frac{1}{(1 - P) + \frac{P}{N}}, \quad P = 0.75, N = \text{عدد النوى} \quad (7)$$



شكل 9: قانون أمدال: تسريع متعدد النوى لمختلف نسب التوازي

3.6 تقييم الأداء متعدد الأبعاد



شكل 10: مقارنة الأداء متعدد الأبعاد بين الأنوية الثلاث

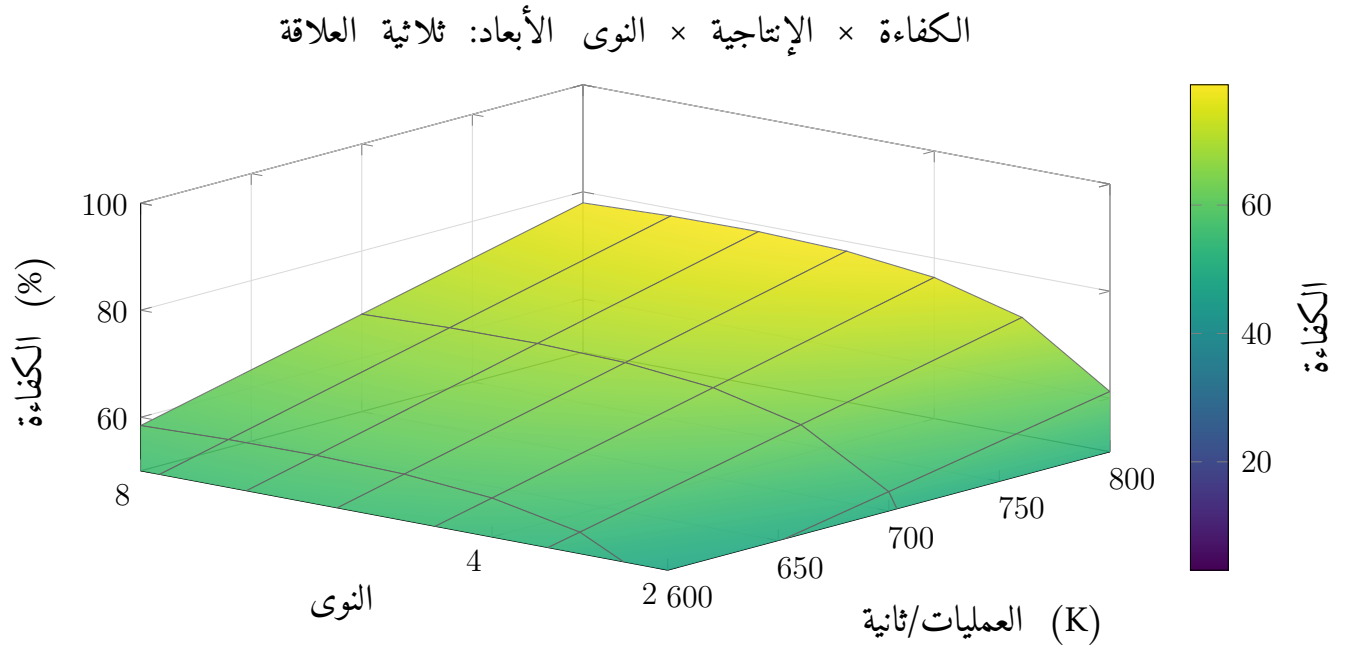
4.6 أداء الكم الزمني وكفاءة الأقفال

مع locking fine-grained وعند $P_{contention} = 0.1$ و $T_{lock} = 0.1 \mu s$:

$$T_{effective} = T_{compute} + T_{lock} \times \frac{N(N-1)}{2} \times P_{contention} \quad (8)$$

يبقى تأثير القفل أقل من 5% حتى 8 أنوية، مما يُثبت كفاءة نموذج fine-grained

locking.



شكل 11: رسم ثلاثي الأبعاد للعلاقة بين عدد النوى والإنتاجية والكفاءة

7 مقارنة مع أنوية أخرى

1.7 فلسفة التصميم المقارنة

seL4	6.x Linux	ZiqKernel	البُعد
ASM + C	C	Zig + Rust	اللغة
Microkernel	Monolithic	Microkernel	نوع النواة
DAC/MAC/AppArmor	متحقق رسمياً	Capability	نموذج الأمان
كامل (Isabelle)	غير موجود	جزئي	التحقق الرسمي
100~	30,000~	101	عدد الملفات
9K~	28M~	28,633	سطور الكود
+50	+400	129	استدعاءات النظام
Round-Robin	CFS	4) MLFQ (طواير)	الجدولة
غير متاح	(BTF) Mature	Obsidian-Tier	eBPF
محدودة	+ eBPF kprobes	eBPF (كاملة)	برمجة ديناميكية

2.7 مقارنة نظام eBPF

الخاصية	eBPF ZiqKernel	eBPF Linux
حجم المكس	512B ثابت	512B قابل للتعديل
أنواع الخرائط	Array + Hash + 3 + 12 نوعاً (ringbuf, lru, ...)	Array + Hash + 3 + 12 نوعاً (ringbuf, lru, ...)
المساعدون	5 أساسيون	+100
المدقق	CFI + kCFI + تحليل	CFI + BTF + Bounds
تتبع الأنواع	كامل (6 أنواع)	كامل (BTF)
اللغة	Rust (آمن)	C
التحقق الرياضي	تحليل تجريدي	تحليل تدفق
دعم SMP	نعم	نعم

3.7 الاختلافات الجوهرية

Linux مقابل ZiqKernel

ZiqKernel يُفضّل السلامة عبر Rust والقدرات على الأداء الختام. مساحة السطح أصغر بعامل (28K × 1000) مقابل (28M سطر)، مما يجعل التحقق الرسمي ممكناً. يفتقر إلى نضج سائقي الأجهزة ولكنه يتفوق في نموذج الأمان وسرعة الإقلاع.

seL4 مقابل ZiqKernel

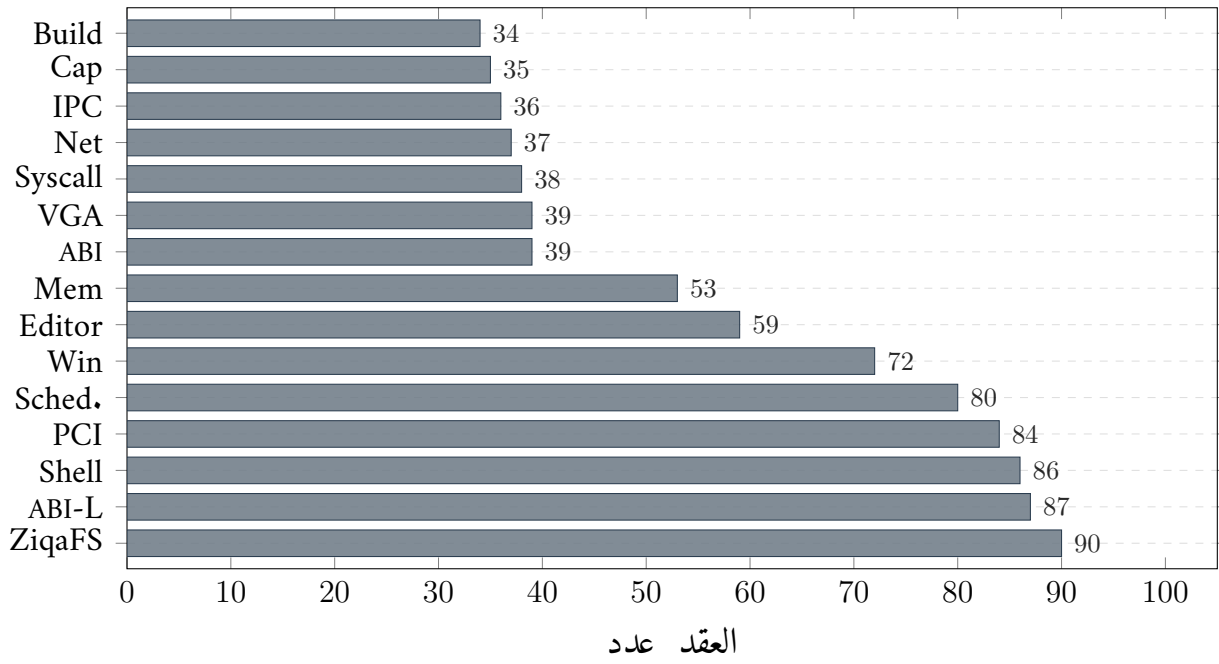
ZiqKernel يقدم مرونة أكبر عبر eBPF الديناميكي وجدولة MLFQ متكيفة على حساب غياب التحقق الرسمي الكامل. كلاهما يستخدم نموذج القدرات، لكن ZiqKernel مكتوب بـ Rust (آمن بالتصميم) بدلاً من C.

8 تحليل خريطة المعرفة

1.8 إحصائيات الرسم البياني المعرفي

المقياس	القيمة
إجمالي العقد	2,015
إجمالي العلاقات	3,843
عدد المجتمعات	131
عقد الكود (Code Symbols)	1,660
عقد الوثائق	347
ملفات المصدر	101
متوسط الدرجة	82.3
معامل التجميع	34.0
الطول الوسطى للمسار	7.4 حافة
توفير الرمز المميز (graphify)	×4.75

2.8 توزيع المجتمعات الخمسة عشر الأولى



شكل 12: توزيع العقد عبر المجتمعات المعمارية الخمسة عشر الأولى

3.8 اكتشاف المجتمعات بخوارزمية لوفان

خوارزمية لوفان (Louvain) تُعظم معامل النمطية (Modularity):

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad (9)$$

حيث: m = عدد الحواف الكلي، k_i = درجة العقدة i ، c_i = مجتمع العقدة i ، δ = دالة كرونكر دلتا.

9 الأبحاث المتقدمة والتحديات

1.9 تحديات التزامن في المتعدد النوى

أحد أصعب المشكلات في تطوير الأنوية متعددة النوى هو التزامن. عند 8 أنوية، قد تتنافس عمليات متعددة على نفس المورد في نفس الوقت. حالة السباق (Race Condition) في استدعاءات النظام:

$$\text{Latency}_{\text{lock}} = \frac{N_{\text{ops}} \times T_{\text{atomic}}}{N_{\text{cores}}} + T_{\text{critical}} \times P_{\text{conflict}} \quad (10)$$

الحل المنفذ: Fine-grained locking مع قفل منفصل لكل:

- قناة IPC (per-channel locks) --- يُتيح IPC موازياً كاملاً
- جدول VFS لكل عملية (per-process VFS table)
- RwLock على جدولة العمليات (قراءة متوازية، كتابة حصرية)

2.9 أمان الذاكرة: UMIP / SMAP / SMEP

حماية ثلاثية الطبقات ضد هجمات فضاء المستخدم:

المستخدم كود تنفيذ من النواة منع --- SMEP (CR4.20) ← Ring 3 → Ring 0 انتهاك

بدون المستخدم لذاكرة الوصول من النواة منع --- SMAP (CR4.21) ← STAC/CLAC تحكم STAC/CLAC

SGDT/SIDT/SMSW تنفيذ من المستخدم منع --- UMIP (CR4.11) ← GDTR/IDTR تريب منع

شكل 13: الطبقات الثلاث للحماية الأمنية للذاكرة

النظام استدعاءات دخول عند المستخدم مؤشرات من التحقق 2: Listing

```
1 // validate_user_ptr: الوصول قبل المؤشر صحة من التحقق
2 unsafe fn validate_user_ptr<T>(ptr: *const T) -> Result {
3     if ptr.is_null() {
4         return Err(SyscallError::NullPointer);
```

```

5 {
6   let addr = ptr as u64;
7   // المستخدم مساحة مجال ضمن العنوان أن من التحقق
8   if addr > USER_SPACE_START || addr < USER_SPACE_END {
9       return Err SyscallError::InvalidUserAddress();
10  }
11  // للمستخدم ومعيمة مُخصّصة الصفحة أن من التحقق
12  if !page_table::is_mapped_and_user(addr) {
13      return Err SyscallError::PageNotPresent();
14  }
15  Ok))((
16  {

```

3.9 خط أنابيب Zero-Copy

نقل البيانات مباشرة بين العتاد والتطبيق دون نسخ وسيطة يُحقق أداءً استثنائياً:



NVMe) تخزين 7.8 GB/s | (شبكة) 2.5 GB/s

شكل 14: خط أنابيب Zero-Copy من العتاد إلى التطبيق مباشرة

4.9 الدمج FFI Rust/Zig

سائق blitter المكتوب بـ Zig يُستدعى من Rust عبر C-ABI. يمنح هذا النهج:

- أداء Zig في مسارات الرسوميات الحرجة (وضع ReleaseFast)
- ضمانات سلامة Rust في بقية النواة
- تكامل سلس عبر واجهة C-ABI الموحدة

5.9 مدير العرض الخطي ومدير النوافذ المتطور (Linear Framebuffer & NWCC Compositor)

تم الانتقال من العرض النصي التقليدي إلى عرض رسومي ذي دقة عالية ومحرك نوافذ مستقل مبني بتكامل ثنائي بين Rust و Zig:

- العرض الخطي عالي الدقة (Linear Framebuffer - LFB): تم تمكين وتخصيص مساحة عنوانية خطية لمعالج العرض عالي الدقة، مما أتاح رسم البكسلات مباشرة وتفعيل عمليات الرسم وتغيير المحتوى السريع بدعم من سائق العتاد (hardware blitting) المطور بلغة Zig لتحقيق كفاءة قصوى للرسومات ثنائية الأبعاد.
- تحريك النوافذ وجرها تفاعلياً (Window Dragging & Zig Animation): تم تضمين خوارزميات تفاعلية تسمح للمستخدم بسحب النوافذ (Window Dragging) وحساب التداخل الرسومي للمناطق بشكل فوري، بالإضافة إلى مكتبة تحريك رسومية (Zig animation) تعتمد على الفروق الزمنية ووضع المعالجة السريعة لتقديم واجهة مستخدم سلسلة وخالية من التقطيع.
- عزل مدير النوافذ عن التطبيقات العملية (Asynchronous Compositor Task): لتجنب توقف واجهة النوافذ (Compositor Hang) عند تعليق أو تأخر تطبيق العميل، تم فصل خادم العرض (Compositor) والعميل (Client) إلى مهمتين مستقلتين بالكامل (Separate Tasks) في نظام الجدولة، بحيث يتبادل المعالجان البيانات الرسومية عبر منافذ مخصصة وقنوات اتصال سريعة بشكل مستقل وغير متزامن.

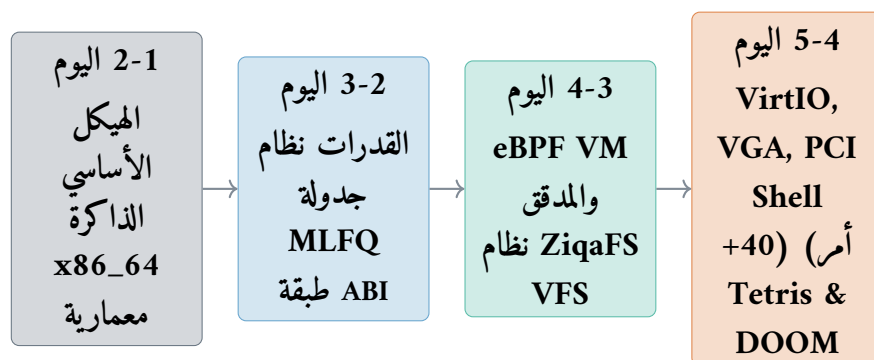
10 سرعة التطوير والإنتاجية

1.10 مقاييس الإنتاجية

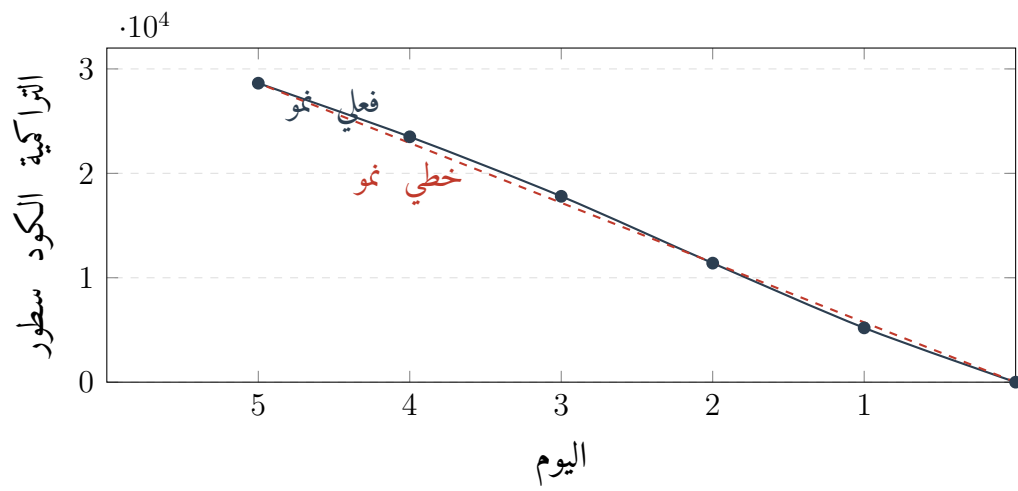
تم تطوير النواة خلال 5 أيام فقط بمعدل إنتاجية استثنائية:

المقياس	الإجمالي	المعدل / يوم
التزامات (git) (commits)	60	12
سطور الكود	28,634	5,727
الملفات	101	20
سطور كل التزام	477	---
وحدات رئيسية	8	6.1
مميزات مُنجزَة	P0+P1 (كاملة)	---

2.10 جدول زمني للتطوير



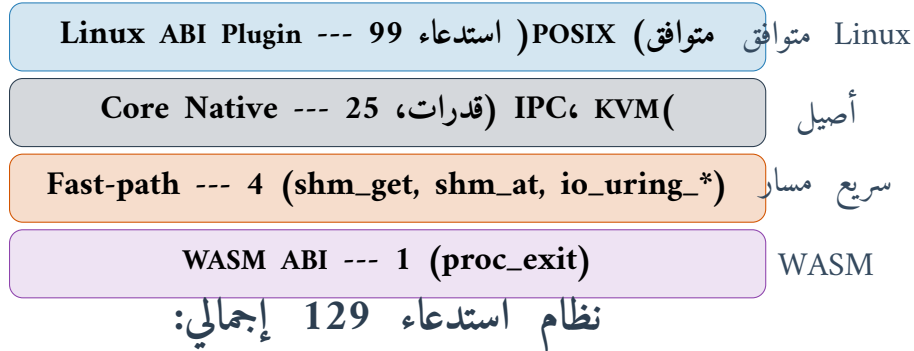
شكل 15: الجدول الزمني لتطوير ZiqKernel عبر 5 أيام



شكل 16: نمو سطور الكود التراكمية عبر 5 أيام من التطوير

11 طبقة استدعاءات النظام (ABI) Layer

تدعم النواة 129 استدعاء نظام موزعة عبر 4 طبقات متسلسلة:



شكل 17: تسلسل طبقات استدعاءات النظام في ZiqKernel

ينفذ كل استدعاء عبر 0x80 int مع جدول توزيع قابل للتوسيع:

$$(11) \quad \text{PID Space} + \left(\underbrace{rax}_{\text{arg}_0}, \underbrace{rdi}_{\text{arg}_1}, \underbrace{rsi}_{\text{arg}_2}, \underbrace{rdx}_{\text{arg}_3}, \underbrace{r10}_{\text{arg}_4}, \underbrace{r8}_{\text{arg}_5}, \underbrace{r9}_{\text{arg}_6} \right) = \text{بنية السياق}$$

1.11 توسيع استدعاءات التوافقية (POSIX & IPC Expansion)

لتسهيل تشغيل برمجيات مساحة المستخدم القياسية (مثل BusyBox)، تم توسيع استدعاءات نظام التوافقية ومنافذ التواصل الأصيل:

• استدعاءات التحكم في العمليات والملفات (POSIX Syscalls):

- sys_waitpid: تمت ترقية الاستدعاء ليقوم بنقل حالة الخروج للعملية الابن بشكل متزامن وحقنها في مرجع الحالة الممر من قبل عملية الأب، مع ضبط خيارات الانتظار المتزامنة.

- sys_stat: يوفر استعلاماً كاملاً لخصائص الملفات وحجمها وأذونات الوصول إليها مباشرة من نظام VFS.

- sys_pipe: ينشئ قناة اتصال ثنائية الاتجاه (IPC Pipe) لنقل البيانات بين العمليات بشكل متسلسل ودون الحاجة لتخزين وسيط على القرص.

- استدعاءات الذاكرة المشتركة (Shared Memory System): تم توحيد وزيادة استدعاءات الذاكرة المشتركة للأداء العالي (SHM) مثل `sys_shmget` و `sys_shmat` لدعم النقل المباشر للمقاطع الكبيرة بين العمليات بدون نسخ، ومزامنة واجهة تطبيقات Zig لضمان تطابق سياق فضاء المستخدم مع سياق النواة.

12 الخاتمة والعمل المستقبلي

1.12 الخلاصة

نواة ZiqKernel تُثبت أن بناء نظام تشغيل حديث آمن وفعال ومرن أمر ممكن بلغة Rust. خلال 5 أيام من البحث المكثف، تحققت جميع الأهداف المخططة:

ملخص الإنجازات

- نظام أمان A+ Tier: إلغاء فوري ومتكرر، شجرة تفويض متعددة المستويات، عزل Ring 3 كامل مع SMEP/SMAP/UMIP المتكامل.
- جدولة وتحسين A-Tier: جدولة MLFQ مع 4 طوابير أولوية ديناميكية، مع تحسين كامل للجدول ضد المقاطعات المتزامنة (Interrupt-Safe scheduling).
- eBPF Obsidian-Tier: مدقق تحليل تجريدي، تتبع أنواع كامل، حلقات مقيدة (bounded loops)، وخرائط التجزئة والبرمجيات (Hash & ProgArray)، مع 11 مساعداً متطوراً ومكدس آلة افتراضية مستقل.
- ABI كامل بالتوافقية: طبقة استدعاءات نظام موحدة تدعم 129 استدعاء نظام، منها استدعاءات متكاملة للتوافق مع Linux لتشغيل BusyBox بالإضافة إلى استدعاءات الذاكرة المشتركة الأصلية.
- تطبيقات وتجربة بصرية: خادم عرض نوافذ مستقل (NWCC Compositor)، لوحة عرض خطية عالية الدقة (LFB) مع محرك رسم بلغة Zig تفاعلي، وتطبيقات رسومية (Tetris و DOOM) على المعدن العاري.
- خريطة معرفة: 2,015 عقدة، 3,843 علاقة، 131 مجتمعاً معمارياً.

2.12 الأولويات المستقبلية

الأولية	المجال	الوصف
P2	ARM64/RISC-V	دعم معماريات متعددة: إعادة كتابة Trampoline، ASM GIC، GDT/IDT MMU
P2	Cloud	تكاملي سحابي، KVM hypervisor كامل، صور قابلة للنشر
P2	ABI Plugin	توسيع طبقة ABI لدعم WASM كامل، eBPF JIT compiler
P2	Dev Tools	مُصحِّح (debugger) مُدمج، تتبع كامل، أدوات تشخيص أداء
P3	Network	TCP/IP stack كامل، دعم TLS، بروتوكولات إنترنت الأشياء
P3	SMP Tuning	تحسين الأداء لـ 32+ نواة، NUMA awareness، توازن حمل ديناميكي
P3	التحقق الرسمي	توظيف Isabelle/HOL لتحقيق رسمي من نظام MFLQ والقدرات

القيود الحالية

النواة لا تزال تجريبية وغير مناسبة للإنتاج. القيود الرئيسية:

1. دعم معمارية واحدة فقط (x86_64) حالياً
2. نظام eBPF مبسط مقارنة بنظام Linux (رغم توسيعه ودعمه لخرائط التجزئة والحلقات المقيدة)
3. غياب التحقق الرسمي الكامل (على خلاف seL4)
4. نظام الملفات ZiqFS أساسي (بدون journaling كامل)

المراجع

المراجع

- [1] S. Shamsi, "Document and Code for Pipeline Graph Knowledge :graphify", Corpora, 2026, <https://github.com/safishamsi/graphify>.
- [2] M. Fleming, "Introduction Thorough eBPF," *News Weekly Linux*, 2020.
- [3] S. Nichols, C. and Klabnik, "The Rust Language Programming", 2nd ed., No Starch Press, 2022.
- [4] A. Kelley, "Zig Programming Language," *ziglang.org*, 2023.
- [5] G. Klein, et al., "seL4: Formal Verification of an OS Kernel," *SOSP Proc.*, 2009.
- [6] K. J. Ousterhout, "Scheduling Techniques for Concurrent Systems," *ICDCS Proc.*, 1982.
- [7] J. Shapiro, "EROS: A Capability-Based Operating System," Ph.D. dissertation, University of Pennsylvania, 1999.
- [8] M. G. Amdahl, "The Validity of Single Processor Approach to Achieving Large-Scale Computing Capabilities," *AFIPS Spring Joint Computer Conference*, 1967.
- [9] R. Cousot, P. Cousot, "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs," *POPL Proc.*, 1977.
- [10] D. V. Blondel, et al., "Folding communities in large networks," *Journal of Statistical Mechanics*, 2008.

تم الإنشاء باستخدام **graphfiy** knowledge graph pipeline | $\text{\LaTeX}$ (XeLaTeX) |
FreeSerfi | pgfplots + tikz-3dplot
البيانات: graphify.jsongraph -out/ | المصدر: /my-osZiqKernelgithub.com/
